

Core Concepts — The Autonomy Spectrum and Multi-Agent Topologies

Part 07 —  Agentic AI Systems · 07-agentic-ai/notes/01-autonomy-spectrum-and-topologies.md

Module 06 built one agent: a loop of `think → act → observe` around a set of tools, with memory to carry state across turns. Everything in that loop is still true here — a "multi-agent system" is just several of those loops connected to each other. What's new is the connection: who talks to whom, in what order, and what happens when one of them is wrong. That's topology, and topology is the subject of this note.

1. The spectrum of autonomy

"Agentic" is not a boolean. It's a dial, and most of the mistakes in this space come from turning it up past where the problem needs it. Four points on the dial, in order of increasing autonomy — and decreasing predictability:

Point on the spectrum	What decides the next step	Concrete example	Failure blast radius
Single LLM call	You, once, per request	Classify a support ticket into 5 categories	One wrong label
Tool-using agent (Module 06)	The model, in a bounded loop, choosing among <i>your</i> tools	ReAct agent that looks up an order, checks a policy doc, drafts a refund	One bad tool call, usually recoverable within the loop
Multi-agent system (this module)	Several models, each with a scoped role, coordinating via a topology	Research system: a lead agent decomposes a question, worker agents search independently, a synthesizer merges results	A bad hand-off or a poisoned sub-result propagates before anyone catches it
Autonomous long-running system	The system, continuously, across sessions, with no human in the immediate loop	A coding agent that runs for hours, or a computer-use agent left to complete a multi-app workflow overnight	Drift accumulates silently; the first sign of trouble might be the final state

Each step right buys you scope and loses you predictability. That trade is not symmetric — predictability degrades *faster* than scope grows, because coordination and time both introduce new ways to fail that don't exist at the point to the left. This is the single most important idea in the module, and it recurs in §6 of [notes/02](#) [07-agentic-ai/notes/02-coordination-control-and-evaluation.md] as "emergent behavior vs. controllability."

Where each point is actually appropriate

- **Single call:** the task is one decision, bounded inputs, no need to consult anything external. Ticket routing, sentiment tagging, a single extraction. If you're chaining single calls with hand-written glue code and no

model-driven branching, that's not agentic at all — it's a **pipeline**, and that's often the right answer (see §5 of notes/02).

- **Tool-using agent:** the task needs a variable number of steps that can't be predicted in advance — "look something up, and depending on what you find, look up something else." One coherent context, one set of tools the model can actually reason about (Module 06's guidance: keep the tool list short and the tool names unambiguous).
- **Multi-agent system:** reach for this only when a specific structural reason applies (§3 of notes/02 gives the checklist). The default assumption should be "no."
- **Autonomous long-running system:** genuinely open-ended goals over hours or days — an overnight migration, a multi-day research task, a computer-use agent completing a workflow across several applications. This is where checkpointing, budget governors, and human-on-the-loop oversight stop being nice-to-haves and become required infrastructure (§5 of notes/02).

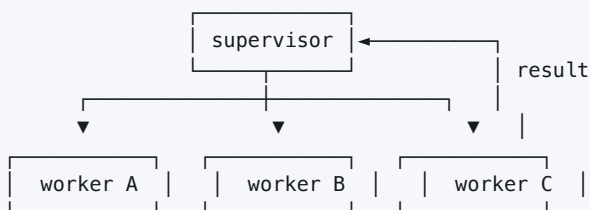
The warning, stated plainly

Do not reflexively maximize autonomy. The pull is real — a more autonomous system feels more impressive in a demo, and "let the agents figure it out" is less design work up front than "I decided the control flow." Both of those are reasons to be suspicious of the instinct, not reasons to follow it. Every notch you turn the dial to the right costs you: more tokens, more latency, more surface area for silent failure, and a harder debugging session at 2am. Pay that cost only when the task's *shape* — not your curiosity, not the framework's marketing — actually requires it. The decision framework in §7 of notes/02 gives you a repeatable way to check.

2. Multi-agent topologies

A topology is the answer to: **who initiates work, who can talk to whom, and who decides when the task is done.** Below are the five shapes that cover essentially every multi-agent system you'll encounter. This is topology *theory* — independent of any framework. Module 16's LangGraph track shows how these compile down to a graph's nodes, edges, and `Command(goto=...)` hand-offs; that's implementation. This is the design decision that precedes it.

2.1 Supervisor / orchestrator-worker



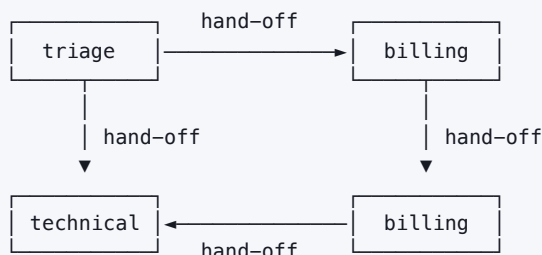
One agent (the supervisor) decomposes the task, routes sub-tasks to specialized workers, and integrates their results. Workers never talk to each other or to the user directly — every hop goes through the supervisor.

- **Problem shape it fits:** a task that decomposes cleanly into independent or loosely-coupled sub-tasks, where you want centralized traceability and per-worker permission scoping. Anthropic's own multi-agent

research system is this shape: a lead agent plans, spins up parallel search sub-agents, and synthesizes.

- **Failure mode:** the supervisor becomes a **context bottleneck** — every worker's output flows back through it, so its own context window fills with summaries of summaries, and it starts making routing decisions on stale or compressed information. It is also a single point of failure: if the supervisor mis-routes, nothing downstream can self-correct without another round trip through it.

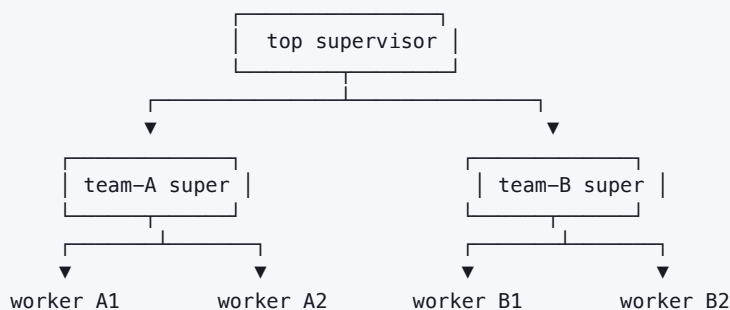
2.2 Peer-to-peer hand-off (swarm)



Any agent can transfer control (and context) directly to any other agent it's aware of — like a call center's warm transfer. There is no central router; control flow is *emergent* from each agent's own decision to hand off.

- **Problem shape it fits:** conversational triage/escalation where the "right" next agent depends on content the current agent just learned — "this is actually a billing issue" — and you want to avoid the extra round trip through a supervisor. Natural fit for support systems with clearly named specialist roles.
- **Failure mode: hand-off loops.** Agent A decides this is really B's problem; B decides, on reflection, that it's actually A's; the user watches two agents pass them back and forth. Because control flow isn't centralized, there's no natural place to detect the loop except a hop counter you have to add yourself. Debugging is harder too — there's no single trace of "the plan," only a chain of local decisions.

2.3 Hierarchical teams-of-teams

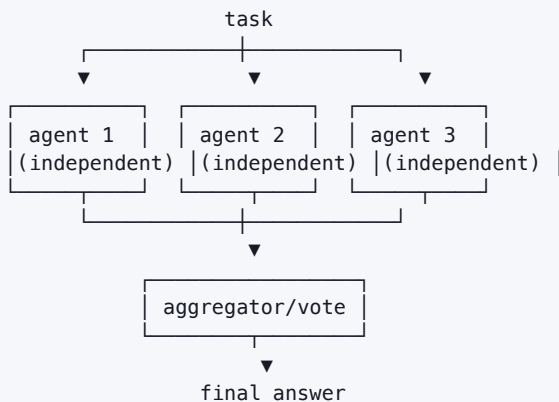


A supervisor of supervisors. Each mid-level supervisor owns a sub-tree the same way a top-level supervisor owns the whole task — recursively the same pattern as §2.1, nested.

- **Problem shape it fits:** genuinely large, org-chart-shaped task trees — a codebase migration split by service, each service owned by a "team" of agents doing analysis/change/test. Scales to task complexity that a flat supervisor can't hold in one context window.

- **Failure mode: latency and cost compound multiplicatively with depth**, and so does the region for the same information-loss problem that afflicts any hand-off (§2 of notes/02) — a detail lost at the leaf has to survive two summarization hops (worker → team supervisor → top supervisor) before it reaches whoever needs it. Debugging requires reconstructing the whole tree, not just one hop.

2.4 Debate / parallel-then-vote



Multiple agents attempt the *same* task independently (ideally with some diversity — different prompts, different models, different retrieved evidence), then a final step aggregates by majority vote, an LLM-judge pick, or a merge. Sometimes the agents see each other's answers for a round of rebuttal before the final vote — genuine "debate" rather than one-shot parallel attempts.

- **Problem shape it fits:** high-stakes single-answer tasks with no natural decomposition, where the value is in *reducing variance* rather than covering more ground — code review, "is this diagnosis plausible," complex reasoning problems where independent attempts sometimes catch each other's mistakes. This is self-consistency (Module 01's "ensembling" and Module 04's sampling-strategies material) promoted to agent-level.
- **Failure mode: correlated errors and thrashing.** If all N agents share the same base model and similar prompts, they tend to fail the same way — voting doesn't fix a shared blind spot, it just launders it with false confidence. In multi-round debate, agents can also fail to converge: two equally confident agents holding opposite positions will burn rounds re-arguing rather than updating, and without an explicit convergence rule (max rounds, a tie-breaking judge) the system doesn't know when to stop.

2.5 Pipeline / assembly-line

```
[research] → [draft] → [fact-check] → [format/style]
```

A fixed sequence of stages, each owned by an agent (or a plain function) specialized for that stage. Unlike supervisor or hand-off patterns, the *order* is not decided at runtime — it's fixed at design time. Only the content flowing through each stage is dynamic.

- **Problem shape it fits:** work that genuinely has stages with different skills/tools/models per stage and a natural forward-only order — write → review → publish; extract → validate → load. This is the topology

that most resembles classic ETL, and that resemblance is a feature: it's the most testable and cheapest of the five because each stage can be evaluated in isolation with fixed inputs.

- **Failure mode: no recovery path backward.** If fact-check finds the draft's central claim is wrong, a rigid pipeline has nowhere to send that information except forward (into formatting) or into a full restart. Pipelines that don't build in an explicit retry/reroute edge silently ship the error, because "stage 4 received bad input" isn't a failure the topology has a way to express.

The five, side by side

Topology	Who decides "next"	Best fit	Signature failure
Supervisor	central router	independent/loosely-coupled sub-tasks, need traceability	supervisor context bottleneck
Peer hand-off	each agent, locally	conversational escalation between named specialists	hand-off loop (deadlock/livelock)
Hierarchical	router of routers	org-chart-shaped, large task trees	cost/latency compound with depth
Debate/vote	an aggregator, after the fact	variance reduction on one hard answer	correlated errors, non-convergence
Pipeline	fixed at design time	staged work with a natural forward order	no backward recovery path

3. Forward / backward links

Idea here	Where it connects
Tool-using agent, ReAct loop	Module 06 [06-ai-agents] — this module assumes it, doesn't re-derive it
Token-cost multiplier, why most systems should stay single-agent	notes/02, §1 [07-agentic-ai/notes/02-coordination-control-and-evaluation.md]
A2A vs. MCP	notes/02, §2 [07-agentic-ai/notes/02-coordination-control-and-evaluation.md] and Module 09 — MCP [09-mcp]
These topologies as LangGraph subgraphs and Command hand-offs	Module 16 — LangGraph track, §5 [16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md]
Emergent behavior vs. controllability, HITL/HOTL	notes/02, §4 [07-agentic-ai/notes/02-coordination-control-and-evaluation.md] and Module 12 — Governance [12-ai-governance]
Evaluating a multi-agent system	notes/02, §6 [07-agentic-ai/notes/02-coordination-control-and-evaluation.md] and Module 15 — AI Evaluation [15-ai-evals]